Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

15 October 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 2-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Superseded results (metric schema 8). The current detector (schema 9) also counts a reload whose address lies inside a wider or merged store (e.g. two 4-byte spills read by one 8-byte load). Schema 8 missed those, so `spill_load`, `spill_event` and the `other_stack` counts here are lower (up to 15% of the spill reloads in bwaves, 8.5% in leela, 3% in x264, under 0.1% in most benchmarks); instruction counts, spill stores and the cache behaviour of the pairs it found are unaffected.

Definitions (metric schema 8). Every stack store issued inside the region of interest, and every later load that reads it back (a load from the same address at the same width), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` ≤ `store` and `load` ≥ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 817 969 294	3 794 919 348	22 956 427	93 519	99.4	21.32	0.10	15.764	64.32	4.455	3.914	0.126	0.159
parest	37 262 401 281	126 952 414	16 148 158	110 058 554	745 702	12.7	0.22	0.59	11.595	57.67	9.586	3.513	2.118	1.649
deepsjeng	46 202 686 169	1 436 088 677	380 604 124	1 046 384 737	9 099 816	26.5	2.78	4.53	9.980	6.89	2.859	7.613	3.282	3.898
leela	26 063 891 644	802 403 734	20 913 175	749 189 404	32 301 155	2.6	0.34	5.75	6.901	20.72	10.600	0.425	1.766	0.859
namd	31 914 751 099	336 698 329	334 454 030	1 939 340	304 959	99.3	3.13	0.01	12.887	25.71	5.486	1.313	3.844	3.891
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	23.735	88.75	0.424	2.261	15.074	0.184
nab	6 691 483 051	150 911 281	124 535 928	24 348 560	2 026 793	82.5	7.90	0.73	6.914	4.63	1.389	2.123	0.898	1.211
xz	2 014 351 521	32 945 274	4 682 897	26 392 608	1 869 769	14.2	0.65	2.62	6.496	25.90	0.133	1.002	2.222	2.241
xz_4m_l1	2 001 462 938	29 215 304	3 958 947	23 548 312	1 708 045	13.6	0.57	2.35	7.996	35.15	0.131	0.808	2.863	1.815
xz_4m_l2	2 083 103 943	28 125 372	3 700 138	22 776 791	1 648 443	13.2	0.51	2.19	9.797	42.07	0.126	0.814	3.396	1.908
xz_4m_l3e	10 573 516 663	124 670 474	54 899 409	66 069 008	3 702 057	44.0	2.44	1.25	6.916	19.19	2.000	1.167	1.747	3.242
xz_4m_l4	4 011 967 585	66 558 242	20 142 751	44 411 548	2 003 943	30.3	2.11	2.21	8.369	30.25	2.282	2.394	1.558	2.249
xz_4m_l4e	12 115 711 352	127 720 342	56 348 941	67 532 844	3 838 557	44.1	2.22	1.11	6.702	17.71	2.107	1.170	1.810	3.475
xz_1m_l0	711 271 503	12 737 212	1 091 050	10 938 543	707 619	8.6	0.56	3.08	5.939	29.24	0.192	2.012	1.021	1.382
xz_1m_l1	719 848 327	12 124 762	973 253	10 458 508	693 001	8.0	0.52	2.91	6.727	33.78	0.193	1.706	1.215	1.441
xz_1m_l2	724 690 222	11 886 991	956 147	10 241 945	688 899	8.0	0.51	2.83	6.928	34.23	0.170	1.718	1.254	1.484
xz_1m_l3e	1 994 406 562	32 343 460	11 257 805	20 010 020	1 075 635	34.8	2.49	2.01	6.214	18.66	1.647	2.131	0.775	2.798
xz_1m_l4	1 076 057 785	22 448 482	5 149 009	16 486 476	812 997	22.9	1.97	3.06	6.832	24.25	1.789	4.355	0.697	2.351
xz_1m_l4e	2 198 787 666	32 786 002	11 469 277	20 222 586	1 094 139	35.0	2.33	1.84	6.173	17.52	1.725	2.109	0.800	2.919
imagick	117 733 663	2 444 391	2 450	2 437 084	4 857	0.1	0.02	4.14	4.422	81.36	10.072	69.568	1.301	0.348
blender	994 421 804	46 392 827	4 430 041	40 844 530	1 118 256	9.5	1.16	8.21	5.341	19.43	2.281	3.165	2.206	2.767
povray	2 240 986 547	168 771 227	2 121 674	155 851 654	10 797 899	1.3	0.27	13.91	10.145	41.21	7.406	20.677	1.842	1.807
omnetpp	12 426 241 547	842 378 057	5 233 032	791 024 854	46 120 171	0.6	0.09	12.73	12.566	26.58	0.250	0.859	1.929	2.480
xalancbmk	395 919 938	18 465 753	295 456	17 282 226	888 071	1.6	0.23	8.73	13.661	33.74	10.003	18.696	3.437	4.325
x264	475 535 613 470	19 188 657 460	13 318 402 468	5 322 257 634	547 997 358	69.4	6.15	2.24	2.326	26.97	0.399	0.954	1.487	1.610
gcc	16 026 870	1 020 580	52 732	952 288	15 560	5.2	0.84	11.88	7.123	32.69	0.717	1.875	1.331	1.231
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	25.499	75.88	5.567	8.282	3.676	5.825
perlbench_makerand	2 421 667 618	224 582 067	55	224 562 979	19 033	0.0	0.08	18.55	5.676	0.00	0.000	2.725	1.717	0.873
bwaves_1	17 288 358 363	145 483 579	145 451 376	26 481	5 722	100.0	3.55	0.00	41.067	31.22	2.704	0.330	4.063	5.189
bwaves_2	17 287 430 556	145 483 497	145 451 376	26 421	5 700	100.0	3.55	0.00	41.069	31.22	2.704	0.330	4.069	5.193
exchange2	57 667 143 431	1 874 700 399	1 304 759 851	345 493 349	224 447 199	69.6	5.03	1.20	2.504	1.08	2.744	3.560	7.229	4.833
fotonik3d	3 098 360 920	28 393 792	27 853 217	437 206	103 369	98.1	3.84	0.03	17.861	41.35	5.709	0.057	1.160	3.747
roms	28 955 766 090	203 000 119	182 774 988	12 238 510	7 986 621	90.0	4.79	0.08	20.998	38.82	4.437	1.564	1.413	2.838
wrf	84 396 898 728	420 492 655	109 777 107	285 402 467	25 313 081	26.1	0.91	0.68	18.055	22.28	9.397	6.610	0.844	1.636
cam4	33 466 803 539	584 634 561	168 239 124	404 426 599	11 968 838	28.8	2.11	2.42	10.243	52.99	2.650	2.913	1.165	1.702
Total	1 003 626 799 442	31 607 978 550	**20 626 590 744**	10 039 378 113	942 009 693	65.3	5.12	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

15 October 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, fully associative L1D/L2 (FALRU) · gem5 `e0a12aa4e3+dirty`

Superseded results (metric schema 8). The current detector (schema 9) also counts a reload whose address lies inside a wider or merged store (e.g. two 4-byte spills read by one 8-byte load). Schema 8 missed those, so `spill_load`, `spill_event` and the `other_stack` counts here are lower (up to 15% of the spill reloads in bwaves, 8.5% in leela, 3% in x264, under 0.1% in most benchmarks); instruction counts, spill stores and the cache behaviour of the pairs it found are unaffected.

Definitions (metric schema 8). Every stack store issued inside the region of interest, and every later load that reads it back (a load from the same address at the same width), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 817 969 294	3 794 919 348	22 956 427	93 519	99.4	21.32	0.10	15.225	59.94	4.848	5.149	0.079	0.113
parest	37 262 401 281	126 952 414	16 148 158	110 058 554	745 702	12.7	0.22	0.59	9.718	68.60	1.355	1.455	0.193	0.415
deepsjeng	46 202 686 169	1 436 088 677	380 604 124	1 046 384 737	9 099 816	26.5	2.78	4.53	8.076	6.11	1.432	7.231	1.581	3.372
leela	26 063 891 644	802 403 734	20 913 175	749 189 404	32 301 155	2.6	0.34	5.75	5.074	27.89	0.357	0.422	0.940	0.358
namd	31 914 751 099	336 698 329	334 454 030	1 939 340	304 959	99.3	3.13	0.01	11.613	28.07	2.563	1.176	3.693	3.758
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.228	99.28	0.000	0.546	15.074	0.184
nab	6 691 483 051	150 911 281	124 535 928	24 348 560	2 026 793	82.5	7.90	0.73	3.889	8.31	0.131	0.108	0.619	0.726
xz	2 014 351 521	32 945 274	4 682 897	26 392 608	1 869 769	14.2	0.65	2.62	5.093	31.78	0.009	0.156	0.181	0.290
xz_4m_l1	2 001 462 938	29 215 304	3 958 947	23 548 312	1 708 045	13.6	0.57	2.35	6.622	41.40	0.010	0.191	0.420	0.300
xz_4m_l2	2 083 103 943	28 125 372	3 700 138	22 776 791	1 648 443	13.2	0.51	2.19	8.610	47.32	0.011	0.205	0.950	0.338
xz_4m_l3e	10 573 516 663	124 670 474	54 899 409	66 069 008	3 702 057	44.0	2.44	1.25	6.036	21.48	0.400	0.295	0.193	1.605
xz_4m_l4	4 011 967 585	66 558 242	20 142 751	44 411 548	2 003 943	30.3	2.11	2.21	6.387	38.80	0.087	0.645	0.496	0.190
xz_4m_l4e	12 115 711 352	127 720 342	56 348 941	67 532 844	3 838 557	44.1	2.22	1.11	5.901	19.54	0.530	0.316	0.198	1.884
xz_1m_l0	711 271 503	12 737 212	1 091 050	10 938 543	707 619	8.6	0.56	3.08	4.615	36.56	0.008	0.492	0.100	0.068
xz_1m_l1	719 848 327	12 124 762	973 253	10 458 508	693 001	8.0	0.52	2.91	5.203	42.80	0.008	0.541	0.162	0.105
xz_1m_l2	724 690 222	11 886 991	956 147	10 241 945	688 899	8.0	0.51	2.83	5.491	42.54	0.008	0.550	0.262	0.112
xz_1m_l3e	1 994 406 562	32 343 460	11 257 805	20 010 020	1 075 635	34.8	2.49	2.01	4.757	23.80	0.235	0.268	0.105	0.750
xz_1m_l4	1 076 057 785	22 448 482	5 149 009	16 486 476	812 997	22.9	1.97	3.06	4.642	34.83	0.018	0.603	0.245	0.089
xz_1m_l4e	2 198 787 666	32 786 002	11 469 277	20 222 586	1 094 139	35.0	2.33	1.84	4.806	21.91	0.340	0.279	0.103	0.892
imagick	117 733 663	2 444 391	2 450	2 437 084	4 857	0.1	0.02	4.14	4.156	87.38	7.202	58.930	0.284	0.302
blender	994 421 804	46 392 827	4 430 041	40 844 530	1 118 256	9.5	1.16	8.21	5.126	19.66	2.125	3.868	2.169	2.989
povray	2 240 986 446	168 771 227	2 121 674	155 851 654	10 797 899	1.3	0.27	13.91	8.700	50.57	7.064	19.522	1.658	0.953
omnetpp	12 426 241 547	842 378 057	5 233 032	791 024 854	46 120 171	0.6	0.09	12.73	11.530	27.53	0.054	0.774	1.152	2.269
xalancbmk	395 919 938	18 465 753	295 456	17 282 226	888 071	1.6	0.23	8.73	13.002	34.56	10.876	21.932	2.438	4.226
x264	475 535 613 470	19 188 657 460	13 318 402 468	5 322 257 634	547 997 358	69.4	6.15	2.24	1.836	31.44	0.157	0.852	0.929	1.181
gcc	16 026 870	1 020 580	52 732	952 288	15 560	5.2	0.84	11.88	5.692	38.27	0.229	1.344	0.406	0.670
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.518	78.48	3.115	6.856	2.578	5.257
perlbench_makerand	2 421 667 618	224 582 067	55	224 562 979	19 033	0.0	0.08	18.55	0.083	0.08	0.000	0.259	0.003	0.002
bwaves_1	17 288 358 363	145 483 579	145 451 376	26 481	5 722	100.0	3.55	0.00	7.312	95.60	0.859	0.001	3.384	5.019
bwaves_2	17 287 430 556	145 483 497	145 451 376	26 421	5 700	100.0	3.55	0.00	7.312	95.60	0.859	0.001	3.391	5.023
exchange2	57 667 143 431	1 874 700 399	1 304 759 851	345 493 349	224 447 199	69.6	5.03	1.20	1.538	1.79	1.696	2.611	3.452	2.322
fotonik3d	3 098 360 920	28 393 792	27 853 217	437 206	103 369	98.1	3.84	0.03	8.886	82.63	1.993	0.006	1.325	3.817
roms	28 955 766 090	203 000 119	182 774 988	12 238 510	7 986 621	90.0	4.79	0.08	10.763	64.31	2.165	0.970	0.303	2.174
wrf	84 396 891 536	420 492 444	109 777 107	285 402 322	25 313 015	26.1	0.91	0.68	5.883	61.76	5.433	4.485	0.483	1.248
cam4	33 466 803 502	584 634 555	168 239 124	404 426 594	11 968 837	28.8	2.11	2.42	8.802	59.48	1.455	2.470	0.630	1.409
Total	1 003 626 792 112	31 607 978 333	**20 626 590 744**	10 039 377 963	942 009 626	65.3	5.12	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

15 October 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, direct-mapped L1D/L2 · gem5 `e0a12aa4e3+dirty`

Superseded results (metric schema 8). The current detector (schema 9) also counts a reload whose address lies inside a wider or merged store (e.g. two 4-byte spills read by one 8-byte load). Schema 8 missed those, so `spill_load`, `spill_event` and the `other_stack` counts here are lower (up to 15% of the spill reloads in bwaves, 8.5% in leela, 3% in x264, under 0.1% in most benchmarks); instruction counts, spill stores and the cache behaviour of the pairs it found are unaffected.

Definitions (metric schema 8). Every stack store issued inside the region of interest, and every later load that reads it back (a load from the same address at the same width), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 817 969 294	3 794 919 348	22 956 427	93 519	99.4	21.32	0.10	17.357	48.80	6.419	5.506	1.503	0.285
parest	37 262 401 281	126 952 414	16 148 158	110 058 554	745 702	12.7	0.22	0.59	15.359	47.58	20.722	6.626	4.432	5.347
deepsjeng	46 202 686 169	1 436 088 677	380 604 124	1 046 384 737	9 099 816	26.5	2.78	4.53	13.686	19.74	6.735	9.452	4.189	4.775
leela	26 063 891 644	802 403 734	20 913 175	749 189 404	32 301 155	2.6	0.34	5.75	8.956	17.75	20.160	0.455	3.522	4.806
namd	31 914 751 099	336 698 329	334 454 030	1 939 340	304 959	99.3	3.13	0.01	15.582	37.33	12.114	2.494	4.501	4.822
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	27.221	75.06	1.630	5.456	15.074	0.184
nab	6 691 483 051	150 911 281	124 535 928	24 348 560	2 026 793	82.5	7.90	0.73	10.786	15.30	1.540	1.648	2.584	2.927
xz	2 014 351 521	32 945 274	4 682 897	26 392 608	1 869 769	14.2	0.65	2.62	8.704	28.28	1.423	0.574	2.159	2.774
xz_4m_l1	2 001 462 938	29 215 304	3 958 947	23 548 312	1 708 045	13.6	0.57	2.35	10.079	36.30	1.566	0.593	2.726	1.808
xz_4m_l2	2 083 103 943	28 125 372	3 700 138	22 776 791	1 648 443	13.2	0.51	2.19	11.652	42.38	1.613	0.611	3.599	2.134
xz_4m_l3e	10 573 516 663	124 670 474	54 899 409	66 069 008	3 702 057	44.0	2.44	1.25	9.401	25.69	3.211	2.596	3.576	3.921
xz_4m_l4	4 011 967 585	66 558 242	20 142 751	44 411 548	2 003 943	30.3	2.11	2.21	10.814	33.53	3.105	3.152	3.148	2.674
xz_4m_l4e	12 115 711 352	127 720 342	56 348 941	67 532 844	3 838 557	44.1	2.22	1.11	9.165	24.56	3.325	2.620	3.659	4.145
xz_1m_l0	711 271 503	12 737 212	1 091 050	10 938 543	707 619	8.6	0.56	3.08	7.980	29.63	2.299	0.928	1.658	1.510
xz_1m_l1	719 848 327	12 124 762	973 253	10 458 508	693 001	8.0	0.52	2.91	8.793	33.78	2.452	0.935	1.717	1.579
xz_1m_l2	724 690 222	11 886 991	956 147	10 241 945	688 899	8.0	0.51	2.83	8.922	34.53	2.470	0.939	1.797	1.667
xz_1m_l3e	1 994 406 562	32 343 460	11 257 805	20 010 020	1 075 635	34.8	2.49	2.01	8.680	23.89	2.864	3.225	1.979	3.759
xz_1m_l4	1 076 057 785	22 448 482	5 149 009	16 486 476	812 997	22.9	1.97	3.06	8.845	28.51	2.999	4.939	1.754	3.271
xz_1m_l4e	2 198 787 666	32 786 002	11 469 277	20 222 586	1 094 139	35.0	2.33	1.84	8.884	22.40	2.964	3.234	2.007	3.876
imagick	117 733 663	2 444 391	2 450	2 437 084	4 857	0.1	0.02	4.14	7.434	47.92	16.948	70.350	13.154	0.436
blender	994 421 804	46 392 827	4 430 041	40 844 530	1 118 256	9.5	1.16	8.21	9.917	31.78	2.962	7.024	5.637	4.108
povray	2 241 026 793	168 772 970	2 121 674	155 853 333	10 797 963	1.3	0.27	13.91	13.662	37.82	13.927	21.862	3.772	4.330
omnetpp	12 426 241 547	842 378 057	5 233 032	791 024 854	46 120 171	0.6	0.09	12.73	15.066	36.61	0.724	10.669	3.331	3.571
xalancbmk	395 919 938	18 465 753	295 456	17 282 226	888 071	1.6	0.23	8.73	15.161	41.74	13.191	18.142	4.760	6.007
x264	475 535 613 470	19 188 657 460	13 318 402 468	5 322 257 634	547 997 358	69.4	6.15	2.24	3.981	26.05	1.001	1.203	1.895	2.419
gcc	16 026 870	1 020 580	52 732	952 288	15 560	5.2	0.84	11.88	10.071	32.52	3.159	4.623	3.612	2.891
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	28.220	72.37	8.955	10.166	5.465	6.451
perlbench_makerand	2 421 667 618	224 582 067	55	224 562 979	19 033	0.0	0.08	18.55	12.684	0.38	0.000	1.444	14.407	5.926
bwaves_1	17 288 358 363	145 483 579	145 451 376	26 481	5 722	100.0	3.55	0.00	44.118	27.18	5.350	1.371	4.864	6.053
bwaves_2	17 287 430 556	145 483 497	145 451 376	26 421	5 700	100.0	3.55	0.00	44.120	27.18	5.350	1.371	4.879	6.060
exchange2	57 667 143 431	1 874 700 399	1 304 759 851	345 493 349	224 447 199	69.6	5.03	1.20	4.008	1.04	3.419	3.666	4.345	3.792
fotonik3d	3 098 360 920	28 393 792	27 853 217	437 206	103 369	98.1	3.84	0.03	22.434	38.96	9.885	0.425	2.099	4.269
roms	28 955 766 090	203 000 119	182 774 988	12 238 510	7 986 621	90.0	4.79	0.08	30.012	38.90	7.109	3.376	3.203	4.921
wrf	84 396 927 626	420 494 362	109 777 107	285 403 859	25 313 396	26.1	0.91	0.68	26.370	26.50	13.692	9.129	4.905	2.812
cam4	33 466 803 465	584 634 549	168 239 124	404 426 589	11 968 836	28.8	2.11	2.42	14.018	44.30	5.423	4.410	2.154	2.157
Total	1 003 626 868 512	31 607 981 988	**20 626 590 744**	10 039 381 174	942 010 070	65.3	5.12	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

15 October 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 4-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Superseded results (metric schema 8). The current detector (schema 9) also counts a reload whose address lies inside a wider or merged store (e.g. two 4-byte spills read by one 8-byte load). Schema 8 missed those, so `spill_load`, `spill_event` and the `other_stack` counts here are lower (up to 15% of the spill reloads in bwaves, 8.5% in leela, 3% in x264, under 0.1% in most benchmarks); instruction counts, spill stores and the cache behaviour of the pairs it found are unaffected.

Definitions (metric schema 8). Every stack store issued inside the region of interest, and every later load that reads it back (a load from the same address at the same width), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` ≤ `store` and `load` ≥ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 817 969 294	3 794 919 348	22 956 427	93 519	99.4	21.32	0.10	15.296	65.69	4.857	4.272	0.084	0.119
parest	37 262 401 281	126 952 414	16 148 158	110 058 554	745 702	12.7	0.22	0.59	10.210	65.62	2.851	1.609	0.422	0.901
deepsjeng	46 202 686 169	1 436 088 677	380 604 124	1 046 384 737	9 099 816	26.5	2.78	4.53	8.904	7.54	2.010	7.255	2.447	3.442
leela	26 063 891 644	802 403 734	20 913 175	749 189 404	32 301 155	2.6	0.34	5.75	5.854	24.20	4.525	0.422	1.494	0.454
namd	31 914 751 099	336 698 329	334 454 030	1 939 340	304 959	99.3	3.13	0.01	11.705	28.28	2.779	1.143	3.769	3.776
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.660	97.36	0.000	0.675	15.074	0.184
nab	6 691 483 051	150 911 281	124 535 928	24 348 560	2 026 793	82.5	7.90	0.73	5.327	6.02	0.506	0.311	0.637	0.759
xz	2 014 351 521	32 945 274	4 682 897	26 392 608	1 869 769	14.2	0.65	2.62	5.694	29.53	0.009	0.207	1.591	1.545
xz_4m_l1	2 001 462 938	29 215 304	3 958 947	23 548 312	1 708 045	13.6	0.57	2.35	7.171	39.21	0.010	0.223	2.255	1.182
xz_4m_l2	2 083 103 943	28 125 372	3 700 138	22 776 791	1 648 443	13.2	0.51	2.19	9.127	45.18	0.011	0.240	3.054	1.179
xz_4m_l3e	10 573 516 663	124 670 474	54 899 409	66 069 008	3 702 057	44.0	2.44	1.25	6.348	20.93	1.083	0.505	0.782	1.822
xz_4m_l4	4 011 967 585	66 558 242	20 142 751	44 411 548	2 003 943	30.3	2.11	2.21	7.394	34.26	0.995	0.992	0.812	0.663
xz_4m_l4e	12 115 711 352	127 720 342	56 348 941	67 532 844	3 838 557	44.1	2.22	1.11	6.199	19.17	1.201	0.518	0.827	2.083
xz_1m_l0	711 271 503	12 737 212	1 091 050	10 938 543	707 619	8.6	0.56	3.08	5.145	33.71	0.008	0.702	0.710	0.671
xz_1m_l1	719 848 327	12 124 762	973 253	10 458 508	693 001	8.0	0.52	2.91	5.814	39.07	0.008	0.667	0.890	0.685
xz_1m_l2	724 690 222	11 886 991	956 147	10 241 945	688 899	8.0	0.51	2.83	6.067	39.07	0.008	0.681	0.961	0.694
xz_1m_l3e	1 994 406 562	32 343 460	11 257 805	20 010 020	1 075 635	34.8	2.49	2.01	5.462	21.23	0.793	0.675	0.307	1.086
xz_1m_l4	1 076 057 785	22 448 482	5 149 009	16 486 476	812 997	22.9	1.97	3.06	5.828	28.43	0.721	1.355	0.357	0.534
xz_1m_l4e	2 198 787 666	32 786 002	11 469 277	20 222 586	1 094 139	35.0	2.33	1.84	5.473	19.76	0.884	0.678	0.318	1.220
imagick	117 733 663	2 444 391	2 450	2 437 084	4 857	0.1	0.02	4.14	4.200	85.57	8.468	68.436	0.324	0.318
blender	994 421 804	46 392 827	4 430 041	40 844 530	1 118 256	9.5	1.16	8.21	5.064	20.38	3.273	3.869	1.857	2.825
povray	2 240 986 446	168 771 227	2 121 674	155 851 654	10 797 899	1.3	0.27	13.91	9.134	45.58	6.987	19.611	1.736	1.170
omnetpp	12 426 241 547	842 378 057	5 233 032	791 024 854	46 120 171	0.6	0.09	12.73	11.852	28.00	0.054	0.819	1.407	2.080
xalancbmk	395 919 938	18 465 753	295 456	17 282 226	888 071	1.6	0.23	8.73	13.280	34.60	9.923	18.315	2.541	4.039
x264	475 535 613 470	19 188 657 460	13 318 402 468	5 322 257 634	547 997 358	69.4	6.15	2.24	1.966	31.00	0.225	0.886	0.994	1.293
gcc	16 026 870	1 020 580	52 732	952 288	15 560	5.2	0.84	11.88	6.093	37.00	0.342	1.531	0.525	0.847
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.847	77.84	4.001	7.208	2.778	5.440
perlbench_makerand	2 421 667 618	224 582 067	55	224 562 979	19 033	0.0	0.08	18.55	3.047	0.00	0.000	0.167	0.003	0.057
bwaves_1	17 288 358 363	145 483 579	145 451 376	26 481	5 722	100.0	3.55	0.00	40.946	31.23	3.813	0.021	3.312	4.917
bwaves_2	17 287 430 556	145 483 497	145 451 376	26 421	5 700	100.0	3.55	0.00	40.947	31.23	3.813	0.021	3.316	4.920
exchange2	57 667 143 431	1 874 700 399	1 304 759 851	345 493 349	224 447 199	69.6	5.03	1.20	2.008	1.35	1.745	2.983	4.998	2.280
fotonik3d	3 098 360 920	28 393 792	27 853 217	437 206	103 369	98.1	3.84	0.03	11.962	61.79	3.816	0.008	0.860	3.556
roms	28 955 766 090	203 000 119	182 774 988	12 238 510	7 986 621	90.0	4.79	0.08	14.936	53.16	3.051	0.793	0.316	2.387
Total	885 763 097 074	30 602 851 334	**20 348 574 513**	9 349 549 047	904 727 774	66.5	5.63	2.11	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

15 October 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 8-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Superseded results (metric schema 8). The current detector (schema 9) also counts a reload whose address lies inside a wider or merged store (e.g. two 4-byte spills read by one 8-byte load). Schema 8 missed those, so `spill_load`, `spill_event` and the `other_stack` counts here are lower (up to 15% of the spill reloads in bwaves, 8.5% in leela, 3% in x264, under 0.1% in most benchmarks); instruction counts, spill stores and the cache behaviour of the pairs it found are unaffected.

Definitions (metric schema 8). Every stack store issued inside the region of interest, and every later load that reads it back (a load from the same address at the same width), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
parest	37 262 401 281	126 952 414	16 148 158	110 058 554	745 702	12.7	0.22	0.59	9.895	67.70	1.417	1.473	0.225	0.627
deepsjeng	46 202 686 169	1 436 088 677	380 604 124	1 046 384 737	9 099 816	26.5	2.78	4.53	8.627	7.70	1.774	7.102	2.089	3.483
leela	26 063 891 644	802 403 734	20 913 175	749 189 404	32 301 155	2.6	0.34	5.75	5.161	27.19	0.646	0.422	1.011	0.368
namd	31 914 751 099	336 698 329	334 454 030	1 939 340	304 959	99.3	3.13	0.01	11.605	28.50	2.551	1.166	3.711	3.765
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.730	97.05	0.000	0.633	15.074	0.184
nab	6 691 483 051	150 911 281	124 535 928	24 348 560	2 026 793	82.5	7.90	0.73	4.074	7.87	0.156	0.129	0.628	0.732
xz	2 014 351 521	32 945 274	4 682 897	26 392 608	1 869 769	14.2	0.65	2.62	5.341	31.37	0.009	0.135	0.455	0.617
xz_4m_l1	2 001 462 938	29 215 304	3 958 947	23 548 312	1 708 045	13.6	0.57	2.35	6.788	41.37	0.010	0.167	0.994	0.605
xz_4m_l2	2 083 103 943	28 125 372	3 700 138	22 776 791	1 648 443	13.2	0.51	2.19	8.809	46.80	0.011	0.178	2.158	0.572
xz_4m_l3e	10 573 516 663	124 670 474	54 899 409	66 069 008	3 702 057	44.0	2.44	1.25	6.094	21.75	0.469	0.362	0.371	1.391
xz_4m_l4	4 011 967 585	66 558 242	20 142 751	44 411 548	2 003 943	30.3	2.11	2.21	6.791	37.27	0.233	0.751	0.552	0.305
xz_4m_l4e	12 115 711 352	127 720 342	56 348 941	67 532 844	3 838 557	44.1	2.22	1.11	5.967	19.86	0.597	0.377	0.399	1.663
xz_1m_l0	711 271 503	12 737 212	1 091 050	10 938 543	707 619	8.6	0.56	3.08	4.777	36.16	0.008	0.402	0.180	0.135
xz_1m_l1	719 848 327	12 124 762	973 253	10 458 508	693 001	8.0	0.52	2.91	5.395	42.00	0.008	0.444	0.308	0.188
xz_1m_l2	724 690 222	11 886 991	956 147	10 241 945	688 899	8.0	0.51	2.83	5.685	41.63	0.008	0.450	0.524	0.185
xz_1m_l3e	1 994 406 562	32 343 460	11 257 805	20 010 020	1 075 635	34.8	2.49	2.01	4.970	23.27	0.290	0.382	0.148	0.706
xz_1m_l4	1 076 057 785	22 448 482	5 149 009	16 486 476	812 997	22.9	1.97	3.06	5.046	32.77	0.094	0.779	0.249	0.180
xz_1m_l4e	2 198 787 666	32 786 002	11 469 277	20 222 586	1 094 139	35.0	2.33	1.84	5.016	21.50	0.391	0.391	0.154	0.842
imagick	117 734 434	2 444 393	2 450	2 437 086	4 857	0.1	0.02	4.14	4.168	86.22	7.194	58.765	0.298	0.307
blender	994 421 929	46 392 827	4 430 041	40 844 530	1 118 256	9.5	1.16	8.21	5.037	20.42	2.141	3.868	2.043	2.995
povray	2 240 986 446	168 771 227	2 121 674	155 851 654	10 797 899	1.3	0.27	13.91	8.697	47.73	6.932	19.449	1.477	1.013
omnetpp	12 426 262 471	842 378 226	5 233 032	791 025 027	46 120 167	0.6	0.09	12.73	11.570	28.59	0.053	0.720	1.230	2.191
xalancbmk	395 919 938	18 465 753	295 456	17 282 226	888 071	1.6	0.23	8.73	13.117	35.01	10.131	18.898	2.513	3.924
x264	475 535 613 470	19 188 657 460	13 318 402 468	5 322 257 634	547 997 358	69.4	6.15	2.24	1.858	32.44	0.161	0.861	0.937	1.233
gcc	16 026 870	1 020 580	52 732	952 288	15 560	5.2	0.84	11.88	5.731	38.81	0.260	1.363	0.432	0.677
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.643	78.48	3.545	6.915	2.675	5.246
perlbench_makerand	2 421 667 618	224 582 067	55	224 562 979	19 033	0.0	0.08	18.55	3.575	0.00	0.000	99.996	0.003	0.111
bwaves_1	17 288 358 363	145 483 579	145 451 376	26 481	5 722	100.0	3.55	0.00	26.018	37.38	5.467	0.021	3.308	4.973
bwaves_2	17 287 430 556	145 483 497	145 451 376	26 421	5 700	100.0	3.55	0.00	26.019	37.38	5.467	0.021	3.304	4.977
exchange2	57 667 143 431	1 874 700 399	1 304 759 851	345 493 349	224 447 199	69.6	5.03	1.20	1.342	2.01	1.432	2.700	3.470	2.262
fotonik3d	3 098 360 920	28 393 792	27 853 217	437 206	103 369	98.1	3.84	0.03	9.207	80.25	2.496	0.006	1.060	3.788
roms	28 955 766 090	203 000 119	182 774 988	12 238 510	7 986 621	90.0	4.79	0.08	11.833	67.05	2.452	0.885	0.302	2.456
Total	837 739 863 934	26 784 882 211	**16 553 655 165**	9 326 592 795	904 634 251	61.8	4.73	2.23	0.000	—	—	—	—	—